LAPORAN PRAKTIKUM

STRUKTUR DATA

“Laporan Praktikum Pekan 3”

Disusun Oleh:

Haykal Muhammad Dahnil

2511532014

Dosen Pengampu : Dr. Wahyudi, S.T, M.T.

Asisten Praktikum : Sherly Sukmadira

DAPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga laporan praktikum **Praktikum Struktur Data** untuk pertemuan pekan ke-3 ini dapat diselesaikan dengan baik.

Laporan ini disusun untuk mendokumentasikan hasil praktikum mengenai struktur data **Stack**. Fokus utama pada pekan ini adalah memahami prinsip *Last In First Out* (LIFO) melalui berbagai metode implementasi, mulai dari penggunaan *library* standar Java, implementasi manual berbasis array, hingga penggunaan Stack pada objek kompleks.

Penulis mengucapkan terima kasih kepada dosen pengampu dan asisten praktikum atas arahannya. Penulis menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran yang membangun sangat diharapkan.

Padang 19 April, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR ii
DAFTAR ISI iii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan Praktikum 1
1.3 Manfaat Praktikum 1
BAB II PEMBAHASAN 3
2.1 Class ArrayList1_2511532014 3
2.2 Class DaftarKata_2511532014 4
2.3 Class DaftarKataDriver_2511532014 5
2.4 Class Mahasiswa_2511532014 6
2.5 Class MahasiswaDriver_2511532014 7
BAB III KESIMPULAN 11
DAFTAR PUSTAKA 12

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam struktur data, **Stack** atau tumpukan adalah sekumpulan data yang penambahan dan penghapusan elemennya hanya dapat dilakukan pada satu ujung yang sama, yang disebut sebagai *Top*. Prinsip kerja Stack dikenal dengan istilah **LIFO (Last In First Out)**, yang berarti data yang terakhir kali dimasukkan akan menjadi data yang pertama kali dikeluarkan.

Pemahaman mengenai Stack sangat penting karena struktur ini digunakan dalam berbagai aplikasi nyata, seperti fitur *Undo/Redo* pada pengolah kata, pembalikan urutan data, hingga evaluasi ekspresi matematika kompleks. Pada praktikum pekan ke-3 ini, dilakukan pengujian Stack melalui berbagai skenario untuk memahami fleksibilitas dan batasan dari masing-masing metode implementasi.

1.2 Tujuan Praktikum

Adapun tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

1. Memahami konsep, karakteristik, dan cara kerja struktur data Stack.
2. Mampu mengimplementasikan operasi dasar Stack seperti push, pop, peek, dan isEmpty.
3. Mampu membuat Stack secara manual menggunakan array statis.
4. Mengimplementasikan Stack menggunakan ArrayList untuk menyimpan objek *custom*.
5. Menerapkan algoritma Stack untuk menyelesaikan masalah evaluasi notasi *postfix*.

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini meliputi:

1. Peningkatan Logika Pemrograman: Melatih cara berpikir sistematis dalam mengelola urutan data berdasarkan prioritas tumpukan.

2. Pemahaman Manajemen Memori: Memahami perbedaan antara penggunaan memori statis (array) dan dinamis (ArrayList/Stack Library) dalam penyimpanan data.
3. Keterampilan Problem Solving: Mahasiswa mampu menggunakan Stack sebagai solusi efisien untuk masalah komputasi tertentu, seperti kalkulator postfix dan pencarian data tanpa merusak struktur asli.
4. Persiapan Struktur Data Lanjut: Menjadi dasar yang kuat sebelum mempelajari struktur data lain yang lebih kompleks seperti Queue, Tree, atau Graph.

BAB II
PEMBAHASAN

2.1 Class LatihanStack_2511532014

2.1.1 Langkah Pengerjaan

1. Menggunakan class Stack bawaan dari java.util.
2. Memasukkan angka 42, -3, dan 17 menggunakan fungsi push().
3. Menghapus elemen teratas dengan pop().

2.1.2 Kode Program

```java
package Pekan3_2511532014;

import java.util.Stack;

public class LatihanStack_2511532014 {
    public static void main(String[] args) {
            Stack<Integer> s = new Stack<Integer>();
            s.push(42);
            s.push(-3);
            s.push(17);
            System.out.println("nilai stack= "+s);
            System.out.println("nilai pop= "+s.pop());
            System.out.println("nilai stack setelah pop= "+s);
            System.out.println("nilai peek= "+s.peek());
            System.out.println("nilai stack setelah peek= "+s);
    }
}

```

2.1.3 Output

```
nilai stack= [42, -3, 17]
nilai pop= 17
nilai stack setelah pop= [42, -3]
nilai peek= -3
nilai stack setelah peek= [42, -3]
```

2.1.4 Analisis

Output membuktikan prinsip LIFO. Angka 17 yang masuk terakhir adalah yang pertama keluar saat di-pop. Metode peek() membuktikan bahwa elemen dapat dilihat tanpa harus dikeluarkan dari tumpukan.

2.2 Class StackArray_2511532014

2.2.1 Langkah Pengerjaan

1. Membuat implementasi manual Stack menggunakan array statis berkapasitas 1000.
2. Mengelola pointer top untuk melacak posisi elemen paling atas.
3. Menambahkan proteksi Stack Overflow dan Stack Underflow.

2.2.2 Kode Program

```java
package Pekan3_2511532014;

public class StackArray_2511532014 {
    static final int MAX_2511532014 = 1000;
    int top;
    int a[] = new int[MAX_2511532014];
    boolean isEmpty_2511532014() {
            return (top < 0);
    }
    StackArray_2511532014(){
            top = -1;
    }
    boolean push_2511532014(int x) {
            if (top >= (MAX_2511532014 - 1)) {
                    System.out.println("Stack Overflow");
                    return false;
            } else {
                    a[++top] = x;
                    System.out.println(x + " dimasukan dalam stack");
                    return true;
            }
    }
    int pop_2511532014() {
            if (top < 0) {
                    System.out.println("Stack Underflow");
                    return 0;
            } else {
                    int x = a[top--];
                    return x;
            }
    }
    int peek_2511532014() {
            if (top < 0) {
                    System.out.println("Stack Underflow");
                    return 0;
            } else {
                    int x = a[top];
                    return x;
            }
    }
    void print_2511532014() {
            for (int i = top; i>-1; i--) {
                    System.out.print(" "+ a[i]);
            }
    }
}
```

2.2.3 Analisis

Implementasi ini menggunakan array statis. Keuntungannya adalah kecepatan akses indeks yang konstan ($O(1)$), namun kekurangannya adalah penggunaan memori yang kaku dan risiko *Overflow*.

2.3 Class StackArrayDriver_2511532014

2.3.1 Langkah Pengerjaan

1. Instansiasi objek dari kelas StackArray_2511532014.
2. Melakukan push data 10, 20, 30.
3. Mencetak status elemen setelah pop.

2.3.2 Kode Program

```java
package Pekan3_2511532014;

public class StackArrayDriver_2511532014 {
    public static void main(String[] args) {
            StackArray_2511532014 s = new StackArray_2511532014();
            s.push_2511532014(10);
            s.push_2511532014(20);
            s.push_2511532014(30);
            System.out.println(s.pop_2511532014() + " dikeluarkan dari stack");
            System.out.println("Element teratas adalah: "+ s.peek_2511532014());
            System.out.println("Element pada stack: ");
            s.print_2511532014();
    }
}

```

2.3.3 Output

```
10 dimasukan dalam stack
20 dimasukan dalam stack
30 dimasukan dalam stack
30 dikeluarkan dari stack
Element teratas adalah: 20
Element pada stack:
 20 10
```

2.3.4 Analisis

Output membuktikan bahwa tumpukan manual bekerja dengan benar. Metode print menampilkan elemen dari yang paling baru (20) ke yang terlama (10), mencerminkan visualisasi tumpukan.

2.4 Class NilaiMaksimum_2511532014

2.4.1 Langkah Pengerjaan

1. Membuat fungsi max menggunakan tumpukan backup.
2. Memindahkan data sambil mencari nilai tertinggi dengan Math.max.
3. Mengembalikan data agar tumpukan asli tetap utuh.

2.4.2 Kode Program

```java
package Pekan3_2511532014;
import java.util.*;

public class NilaiMaksimum_2511532014 {
    public static int max_2511532014(Stack<Integer> s) {
            Stack<Integer> backup = new Stack<Integer>();
            int maxValue = s.pop();
            backup.push(maxValue);
            while (!s.isEmpty()) {
                    int next = s.pop();
                    backup.push(next);
                    maxValue = Math.max(maxValue, next);
            }
            while (!backup.isEmpty()) {
                    s.push(backup.pop());
            }
            return maxValue;
    }

    public static void main(String[] args) {
            Stack<Integer> s = new Stack<Integer>();
            s.push(70);
            s.push(12);
            s.push(20);
            System.out.println("isi stack " + s);
            System.out.println("Stack teratas "+ s.peek());
            System.out.println("Nilai maksimum "+ max_2511532014(s));

    }

}

```

2.4.3 Output

```
isi stack [70, 12, 20]
Stack teratas 20
Nilai maksimum 70
```

2.4.4 Analisis

Karena Stack hanya mengizinkan akses elemen teratas, pencarian nilai di tengah memerlukan pemindahan data sementara. Tumpukan backup sangat krusial di sini agar data asli tidak hilang secara permanen.

2.5 Class StackPostfix_2511532014

2.5.1 Langkah Pengerjaan

1. Menerima string notasi postfix.
2. Memasukkan angka ke stack. Saat bertemu operator, ambil dua angka terakhir untuk dioperasikan.

2.5.2 Kode Program

```
package Pekan3_2511532014;

import java.util.Scanner;
import java.util.Stack;

public class StackPostfix_2511532014 {
	public static int postfixEvaluate_2511532014(String expression) {
		Stack<Integer> s = new Stack<Integer>();
		Scanner input = new Scanner(expression);
		while (input.hasNext()) {
			if (input.hasNextInt()) {		// an operand (integer)
				s.push(input.nextInt());
			} else {	//an operator
				String operator = input.next();
				int operand2= s.pop();
				int operand1= s.pop();
				if (operator.equals("+")) {
					s.push(operand1 + operand2);
				} else if (operator.equals("-")) {
					s.push(operand1 - operand2);
				} else if (operator.equals("*")) {
					s.push(operand1 * operand2);
				} else {
					s.push(operand1/operand2);
				}
			}

		}
		input.close();
		return s.pop();
	}

	public static void main(String[] args) {
		System.out.println("Hasil postfix= " + postfixEvaluate_2511532014("5 2 4 * + 7 -"));

```

```
    }

}
```

2.5.3 Output

```
Hasil postfix= 6
```

2.5.4 Analisis

Stack sangat efektif untuk notasi postfix karena urutan masuk operan. Penting untuk dicatat bahwa operan kedua (operand2) diambil lebih dulu agar operasi pengurangan atau pembagian tetap akurat.

2.6 Class SiswaStack_2511532014

2.6.1 Langkah Pengerjaan

1. Membuat kelas objek Siswa_2511532014.
2. Membuat Stack dinamis menggunakan ArrayList.
3. Menjalankan operasi push, pop, dan peek pada objek siswa.

2.6.2 Kode Program

```java
package Pekan3_2511532014;
import java.util.ArrayList;
class Siswa_2511532014 {
    String nama;
    int nim;

    public Siswa_2511532014(String nama, int nim) {
        this.nama = nama;
        this.nim = nim;
    }

    @Override
    public String toString() {
        return "Nim: " + nim + ", Nama: " + nama;
    }
}
public class SiswaStack_2511532014 {
    private ArrayList<Siswa_2511532014> stack;
    public SiswaStack_2511532014() {
        stack = new ArrayList<>();
    }
    public void push_2511532014(Siswa_2511532014 mhs) {
        stack.add(mhs);
    }
    public Siswa_2511532014 pop() {
```

```java
            if (!isEmpty()) {
                return stack.remove(stack.size()-1);
            }
            return null;
    }

    public Siswa_2511532014 peek() {
            if (!isEmpty()) {
                return stack.get(stack.size()-1);
            }
            return null;
    }

    public boolean isEmpty() {
            return stack.isEmpty();
    }

    public void tampilkanSiswa_2511532014() {
            for (int i = stack.size() - 1; i>=0; i--) {
                System.out.println(stack.get(i));
            }
    }

    public static void main(String[] args) {
            SiswaStack_2511532014 studentStack = new SiswaStack_2511532014();

            Siswa_2511532014 mhs1= new Siswa_2511532014("Ali", 1);
            Siswa_2511532014 mhs2= new Siswa_2511532014("Boby", 2);
            Siswa_2511532014 mhs3= new Siswa_2511532014("Charles", 3);

            studentStack.push_2511532014(mhs1);
            studentStack.push_2511532014(mhs2);
            studentStack.push_2511532014(mhs3);

            System.out.println("Siswa di dalam stack:");
            studentStack.tampilkanSiswa_2511532014();

            System.out.println("Siswa teratas "+ studentStack.peek());
            System.out.println("menegeluarkan siswa teratas dari stack: "+ studentStack.pop());
            System.out.println("daftar siswa setelah di pop: ");
            studentStack.tampilkanSiswa_2511532014();
    }

}
```

2.6.3 Output

```
Siswa di dalam stack:
```

```
Nim: 3, Nama: Charles
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
Siswa teratas Nim: 3, Nama: Charles
menegeluarkan siswa teratas dari stack: Nim: 3, Nama: Charles
daftar siswa setelah di pop:
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
```

2.6.4 Analisis

pada program ini menunjukkan fleksibilitas Stack untuk menampung data non-primitif (objek). Iterasi terbalik pada fungsi tampilkan memastikan data tersaji sesuai perspektif tumpukan.

BAB III
KESIMPULAN

Berdasarkan praktikum pekan ke-3, dapat disimpulkan bahwa:

1. **Stack** adalah struktur data linear dengan mekanisme LIFO (*Last In First Out*).
2. Implementasi Stack dapat dilakukan secara manual (array statis) atau menggunakan *Java Library* dan ArrayList (dinamis).
3. Operasi utama Stack adalah push (tambah), pop (hapus dan ambil), serta peek (lihat).
4. Stack memiliki peran penting dalam evaluasi ekspresi postfix dan manajemen data bertingkat.

DAFTAR PUSTAKA

Laporan Tugas

1. Class Website_2511532014

a. Langkah Pengerjaan

i. Mendefinisikan atribut judul_2014, penyanyi_2014, dan durasi_2014 dengan akses private untuk menerapkan prinsip enkapsulasi.

ii. Mendefinisikan atribut private String judul_2014 dan private String url_2014 untuk menyimpan informasi halaman web.

iii. Membuat konstruktor untuk menginisialisasi objek Website_2511532014 saat instansiasi dilakukan.

iv. Menyediakan metode Getter (getJudul_2014, getUrl_2014) dan Setter (setJudul_2014, setUrl_2014) untuk aksesibilitas atribut yang terenkapsulasi.

v. Melakukan override pada metode toString() untuk memformat tampilan data website dalam bentuk teks yang representatif.

b. Kode Program

```java
package Pekan3_2511532014;

public class Website_2511532014 {
        private String judul_2014;
        private String url_2014;

        //konstruktor
        public Website_2511532014(String judul_2014, String url_2014){
                this.judul_2014 = judul_2014;
                this.url_2014 = url_2014;
        }

        //Getter
        public String getJudul_2014() {
                return judul_2014;
        }
        public String getUrl_2014() {
                return url_2014;
        }

        //Setter
        public void setJudul_2014(String judul_2014) {
                this.judul_2014 = judul_2014;
        }
        public void setUrl_2014(String url_2014) {
                this.url_2014 = url_2014;
        }

        @Override
        public String toString() {
                return "Judul: " + judul_2014 + ", URL: " +url_2014;
        }

```

```
}

```

c. **Analisis**

Kelas Website_2511532014 berperan sebagai entitas data tunggal dalam sistem simulasi *browser history*. Analisis pada struktur ini menunjukkan penerapan **Enkapsulasi** yang ketat, di mana variabel *member* bersifat private dan hanya dapat diakses melalui metode publik. Hal ini menjamin integritas data halaman web agar tidak dapat diubah secara ilegal dari luar kelas. Penggunaan @Override toString() memungkinkan objek ini diperlakukan secara transparan saat dicetak, sehingga memudahkan proses *debugging* dan visualisasi data dalam struktur Stack.

2. **Class Browser_2511532014**

a. **Langkah Pengerjaan**

i. Mengimpor kelas java.util.Stack sebagai penampung riwayat dan EmptyStackException untuk penanganan error.

ii. Membuat metode tampilkanMenu_2014() untuk antarmuka pengguna berbasis teks.

iii. Mengimplementasikan metode kunjungiWebsite_2014() yang menggunakan perintah stack.push() untuk menambah halaman baru ke puncak riwayat.

iv. Mengimplementasikan metode tombolBack_2014() menggunakan perintah stack.pop() untuk kembali ke halaman sebelumnya dengan blok try-catch guna menangani kondisi riwayat kosong.

v. Mengimplementasikan metode checkHalamanAktif_2014() menggunakan stack.peek() untuk melihat halaman yang sedang dibuka tanpa menghapusnya.

vi. Menggunakan metode main dengan struktur do-while dan switch-case untuk menjalankan alur logika simulasi.

b. **Kode Program**

```
package Pekan3_2511532014;

import java.util.EmptyStackException;
import java.util.Scanner;
import java.util.Stack;

public class Browser_2511532014 {
        //Menu
        public static void tampilkanMenu_2014() {
                System.out.println("\n=== Browser History NIM:2511532014 ===");
                System.out.println("1. Kunjungi Website");
                System.out.println("2. Tombol Back");
```

```
            System.out.println("3. Lihat Halaman Aktif");
            System.out.println("4. Cek Status History");
            System.out.println("5. Keluar");
        }

        //push
        public static void kunjungiWebsite_2014(Stack<Website_2511532014> stack, Scanner sc) {
            System.out.print("Masukkan Judul: ");
        String judul = sc.nextLine();
        System.out.print("Masukkan URL: ");
        String url = sc.nextLine();
        Website_2511532014 websitebaru = new Website_2511532014(judul, url);
        stack.push(websitebaru);
        System.out.println("Berhasil mengunjungi halaman!");
        }

        //pop
        public static void tombolBack_2014(Stack<Website_2511532014> stack) {
            try {
           Website_2511532014 dihapus = stack.pop();
           System.out.println("Kembali ke halaman sebelumnya.");
           System.out.println("Halaman yang dihapus:");
           System.out.println(dihapus);
        } catch (EmptyStackException e) {
           System.out.println("Error: History kosong! Tidak ada halaman yang bisa di-back.");
        }
        }

        //peek
        public static void checkHalamanAktif_2014(Stack<Website_2511532014> stack) {
            if (!stack.isEmpty()) {
           System.out.println("Halaman Saat Ini:");
           System.out.println(stack.peek());
        } else {
           System.out.println("Tidak ada halaman yang sedang aktif (History kosong).");
        }
        }

        //cek Status
        public static void cekStatus_2014(Stack<Website_2511532014> stack) {
            System.out.println("Jumlah riwayat: " + stack.size());
        System.out.println("Apakah history kosong? " + (stack.isEmpty() ? "Ya" : "Tidak"));
        }

```

```java
    public static void main(String[]args) {
        Scanner scanner = new Scanner(System.in);
        Stack<Website_2511532014> stack_2014 = new Stack<>();
        int pilihan;
        do {
            tampilkanMenu_2014();
            System.out.print("Pilih menu: ");
            pilihan = scanner.nextInt();
            scanner.nextLine();
            System.out.println("");

            switch(pilihan) {
            case 1:
                kunjungiWebsite_2014(stack_2014, scanner);
                break;
            case 2:
                tombolBack_2014(stack_2014);
                break;
            case 3:
                checkHalamanAktif_2014(stack_2014);
                break;
            case 4:
                cekStatus_2014(stack_2014);
                break;
            case 5:
                System.out.println("Terima kasih telah menggunakan simulasi Browser History.");
            default:
                System.out.println("Pilihan tidak valid!");
            }
        } while (pilihan != 5);
        scanner.close();
    }
}
```

c. Output

```
=== Browser History NIM:2511532014 ===
1. Kunjungi Website
2. Tombol Back
3. Lihat Halaman Aktif
4. Cek Status History
5. Keluar
Pilih menu: 1

Masukkan Judul: unand
Masukkan URL: unand.com
Berhasil mengunjungi halaman!

=== Browser History NIM:2511532014 ===
1. Kunjungi Website
2. Tombol Back
```

```
3. Lihat Halaman Aktif
4. Cek Status History
5. Keluar
Pilih menu: 2

Kembali ke halaman sebelumnya.
Halaman yang dihapus:
Judul: unand, URL: unand.com

=== Browser History NIM:2511532014 ===
1. Kunjungi Website
2. Tombol Back
3. Lihat Halaman Aktif
4. Cek Status History
5. Keluar
Pilih menu: 5

Terima kasih telah menggunakan simulasi Browser History.
```

d. Analisis

Program ini merupakan contoh aplikasi praktis dari struktur data Stack dalam mereplikasi fitur nyata, yaitu navigasi *back* pada peramban web. Analisis teknis menunjukkan bahwa setiap kali pengguna mengunjungi website baru, halaman tersebut diletakkan di puncak tumpukan (*Top*) menggunakan push. Saat tombol *Back* (fungsi pop) ditekan, halaman aktif yang berada di puncak tumpukan dibuang, sehingga halaman yang dikunjungi sebelumnya otomatis menjadi halaman aktif yang baru. Hal ini sesuai dengan prinsip **LIFO (Last In First Out)**.

Keunggulan program ini terletak pada penggunaan **Exception Handling** (try-catch) pada metode tombolBack_2014(). Tanpa penanganan EmptyStackException, program akan mengalami *crash* apabila pengguna menekan tombol *back* pada saat riwayat sudah kosong. Selain itu, penggunaan metode peek() pada checkHalamanAktif_2014() adalah keputusan efisien untuk memantau status sistem tanpa memodifikasi atau merusak tumpukan riwayat yang ada.